Compiler Construction Lab - Assignment 6

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

Design a Syntax Analyzer for Arithmetic Expressions.

Use ply.lex and ply.yacc

Read an expression that contains numbers(integers), operators(binary operators +,-,*,/).

Check the syntax.

Generate the appropriate error messages if the expression is invalid.

If the expression is valid then generate the result.

Expected output:

input --> 2+3+ output --> Syntax error at EOF

input --> a+3 output --> Illegal character 'a' at line 1, position 0

input --> 2+3 output --> 5

Theory

A syntax analyzer is a phase of a compiler that checks whether the sequence of tokens generated by the lexical analyzer follows the grammar of a programming language. In this assignment, the syntax analyzer validates arithmetic expressions containing integer operands and operators such as `+`, `-`, `*`, and `/`.

Flex is used to perform lexical analysis. It identifies numbers, operators, parentheses, whitespace, and invalid characters. The recognized tokens are passed to Bison, which performs syntax analysis using grammar rules.

Bison uses context-free grammar to determine whether an expression is syntactically valid. Operator precedence and associativity are specified so that multiplication and division are evaluated before addition and subtraction. Parentheses can be used to change the normal order of evaluation.

The grammar supports:

- Integer operands
- Addition, subtraction, multiplication, and division
- Parenthesized expressions
- Unary minus
- Syntax-error detection

- Invalid-character detection
- Division-by-zero handling

Screenshots and Execution Results

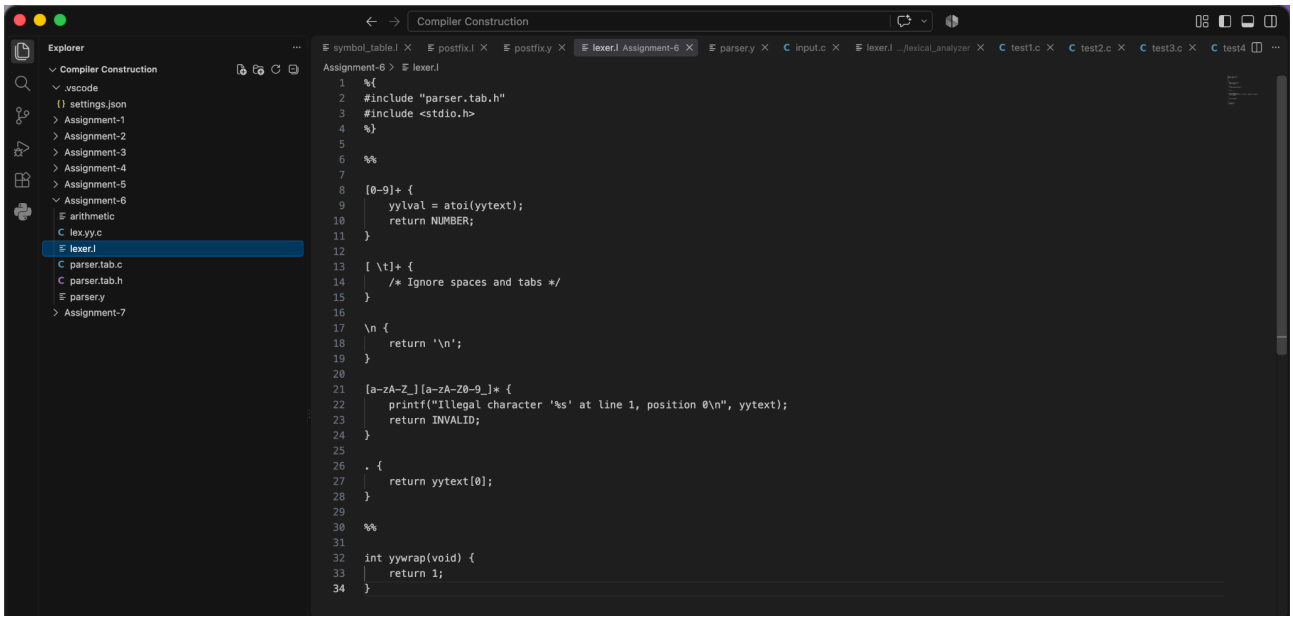


Figure 1: Lexical analyzer implementation in lexer.l

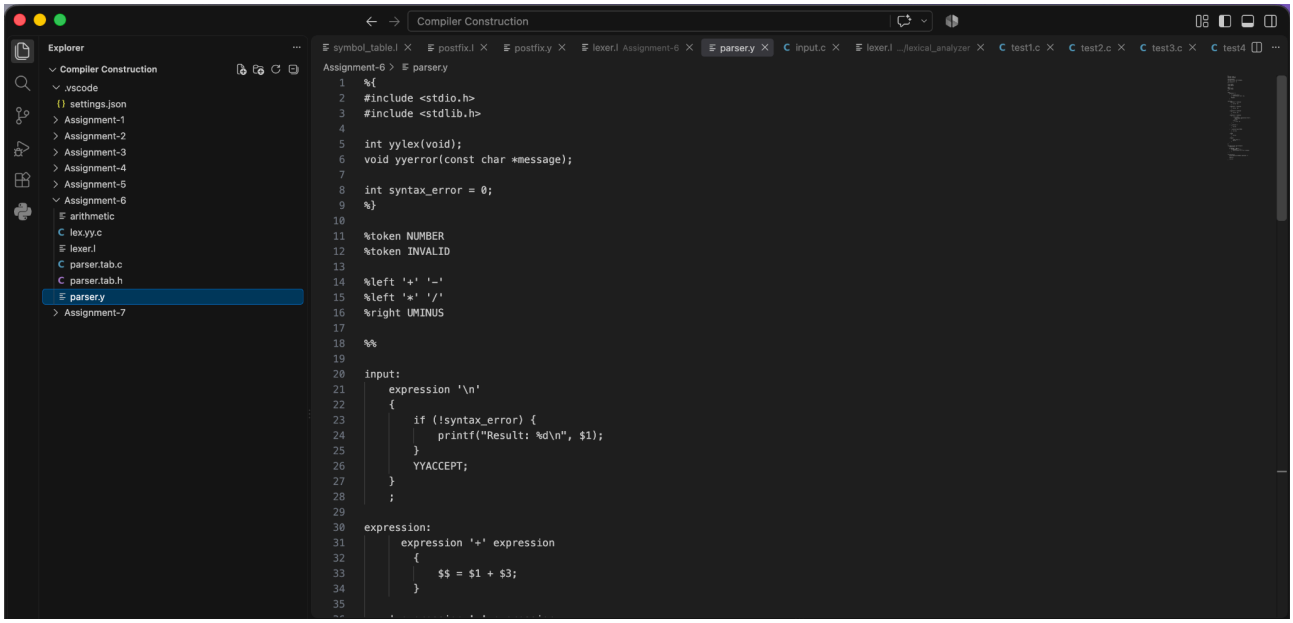


Figure 2: Parser implementation in parser.y — grammar and arithmetic operations

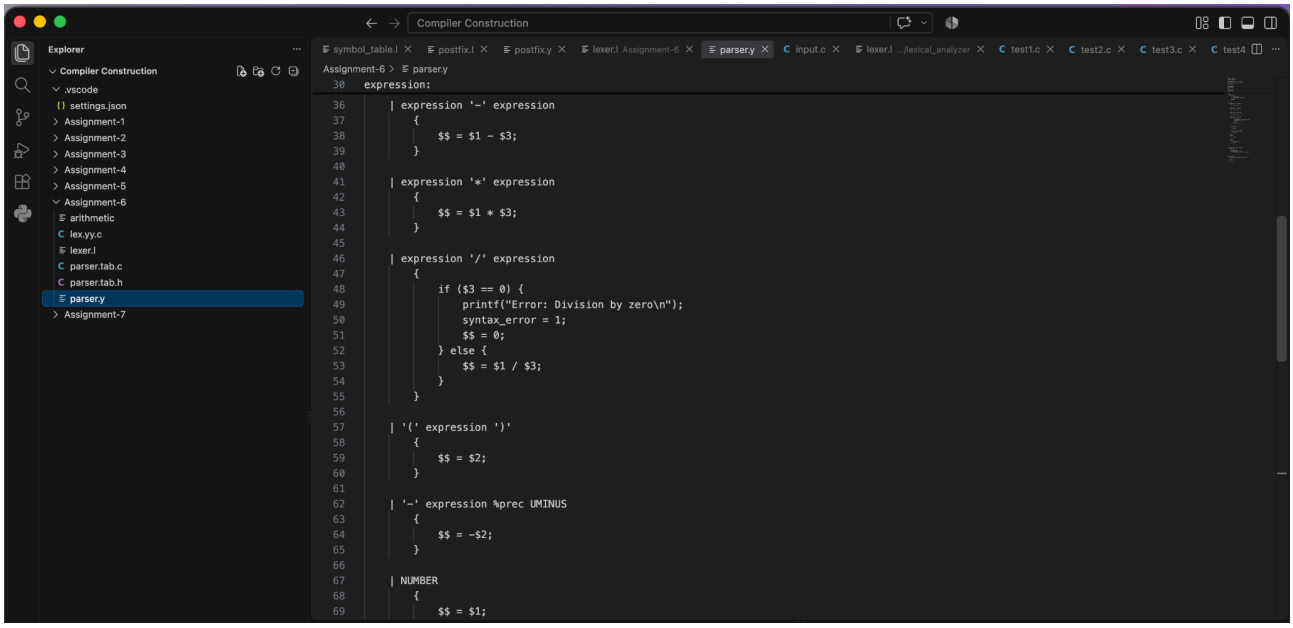


Figure 3: Parser implementation in parser.y — remaining grammar rules and error handling

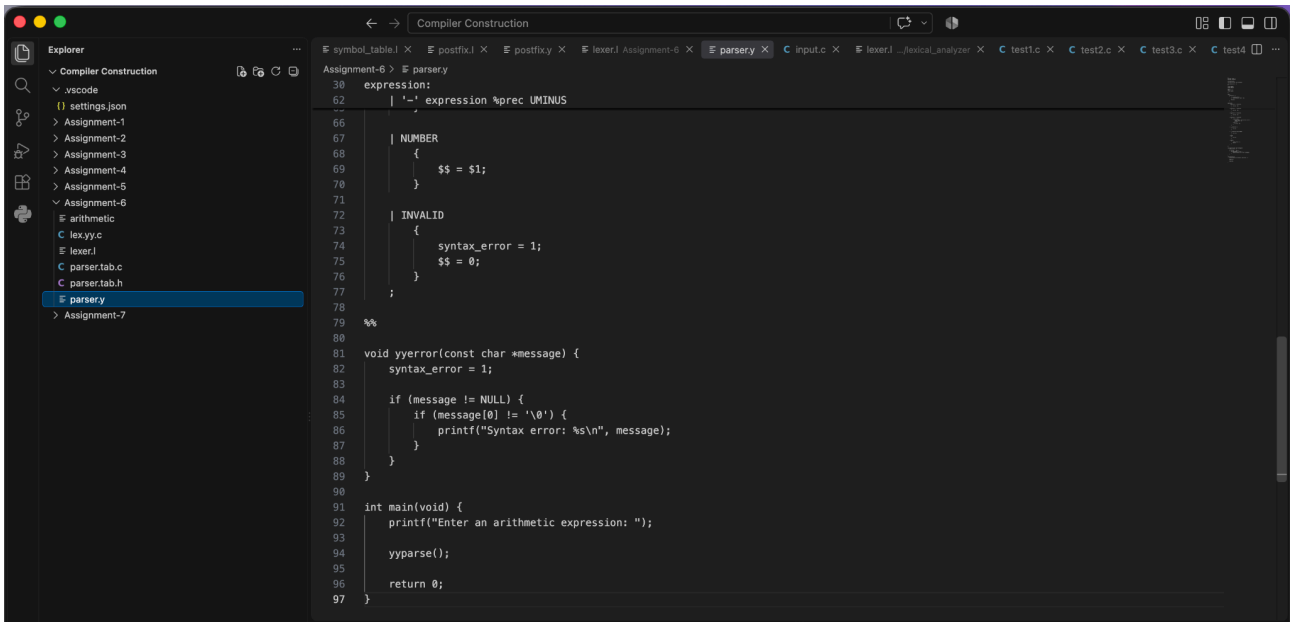


Figure 4: Successful compilation and execution with valid and invalid test cases

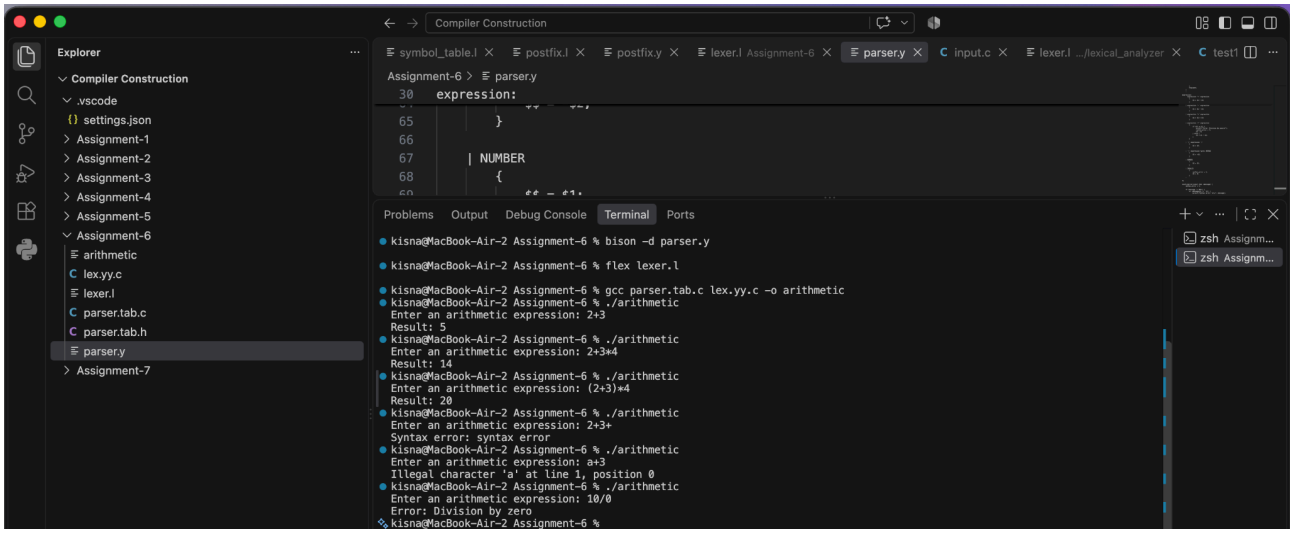


Figure 5: Additional view of the parser implementation

Test Cases Demonstrated

Input	Expected/Observed Result	Purpose
2+3	Result: 5	Valid expression
2+3*4	Result: 14	Operator precedence
(2+3)*4	Result: 20	Parentheses
2+3+	Syntax error	Incomplete expression
a+3	Illegal character 'a'	Invalid identifier
10/0	Error: Division by zero	Runtime arithmetic error

Conclusion

The syntax analyzer was successfully implemented using Flex and Bison. The program evaluates valid arithmetic expressions according to operator precedence and reports errors for incomplete expressions, illegal characters, and division by zero.